ETC3250/5250 Introduction to Machine Learning

Week 4: Logistic regression

Jack Jewson

Jack.Jewson@monash.edu

Department of Econometrics and Business Statistics

ETC3250/5250 Lecture 4

ETC3250/5250 Lecture 4

Overview

We will cover:

- A final kNN model
- Modelling classification probabilities using logistic curves
- What can go wrong in high-d, and how to adjust using **regularisation** methods.

ETC3250/5250 Lecture 4

k-nearest neighbours

ETC3250/5250 Lecture 4

Recap

- Specify a distance metric $d(\cdot, \cdot)$ and a number of neighbours K
- When predicting new observation x'
  - for $i = 1, \ldots, n$ evaluate $d(x', x_i)$
  - select the K observations with the smallest $d(x', x_i)$
  - Predict the response as the average of the y_i of the K nearest neighbours
    - In regression predict $\frac{1}{K} \sum_{k=1}^{K} y_{n_k}$
    - In classification predict $P(y = C) = \frac{1}{K} \sum_{k=1}^{K} \mathbb{I}(y_{n_k} = C)$

Choose K via cross-validation

ETC3250/5250 Lecture 4

How about this penguin?

Whats going on here?

ETC3250/5250 Lecture 4

Standardisation

- The scale of the y-axis (`bd`) is much bigger than the x-axis (`bl`)
- This changes how these variables are weighted when we look for the neighbours
- Closeness in `bl` is upweighted relative to `bd`
- By default you should standardise your variables

```
p_std <- p_tidy |>
  mutate_if(is.numeric, function(x) (x-mean(x))/sd(x))
```

WARNING: This gives all of you features the same weight, and kNN has no other way to determine whether a feature is useful or not

ETC3250/5250 Lecture 4

Weighting Distance

- What class would $K = 3$ kNN predict the circled observation to be?
  - $P(y = A) = 1/3$
  - $P(y = B) = 2/3$
- But the Class A observation was much closer to the observation we want to predict than the Class B observations

Inverse Distance Weighted kNN

- Rather than just predicting the response as the average of the response of the nearest neighbours, we can take a weighted average
- And we weight by 'inverse distance' - $w_k = 1/d(x', x_k)$
  - In regression predict $\sum_{k=1}^{K} \frac{w_{n_k}}{\sum_{k'=1}^{K} w_{n'_k}} y_{n_k}$
  - In classification predict $P(y = C) = \sum_{k=1}^{K} \frac{w_{n_k}}{\sum_{k'=1}^{K} w_{n'_k}} \mathbb{I}(y_{n_k} = C)$

ETC3250/5250 Lecture 4

Inverse Distance Weighted kNN

- In the example
  - The Class A neighbour is distance $\sqrt{(0-\frac{1}{\sqrt{2}})^2+(0-\frac{1}{\sqrt{2}})^2} = 1$ so $w_{n_1} = \frac{1}{1}$
  - The first Class B neighbour is distance $\sqrt{(0-2)^2+(0-2)^2} = \sqrt{8}$ so $w_{n_2} = \frac{1}{\sqrt{8}}$
  - The second Class B neighbour is distance $\sqrt{(0-2)^2+(0-3)^2} = \sqrt{13}$ so $w_{n_2} = \frac{1}{\sqrt{13}}$
- Therefore
  - $P(y = A) = \frac{1}{1+0.35+0.28} = 0.61$
  - $P(y = B) = \frac{0.35+0.28}{1+0.35+0.28} = 0.39$

ETC3250/5250 Lecture 4

Inverse Distance Weighted kNN in `tidymodels`

NOT weighting by distance

```
knn_spec <- nearest_neighbor(neighbors = K, weight_func =
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

Weighting by distance

```
knn_spec <- nearest_neighbor(neighbors = K, weight_func =
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

In general, you can cross-validate whether to weight by distance or not

ETC3250/5250 Lecture 4

Different distances

- You also have flexibility to use different distances
- The kknn package uses the general Minkowski distance $d(x_1, x_2) = \left(\sum_{j=1}^{r} (x_{1j} - x_{2j})^p\right)^{1/p}$ (Note: here is use r for the dimension of the data to avoid confusion $x_1, x_2 \in \mathbb{R}^r$)
- This has a hyperparameter p
- $p = 2$ is the Euclidean distance we were using before
- $p = 1$ is the Manhattan or Absolute distance, gives less penalty to large distances
-
  - $p = 3$ is a Cubic distance, gives greater penalty to large distances

```
knn_spec <- nearest_neighbor(neighbors = K, weight_func =
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

ETC3250/5250 Lecture 4

In general, you can cross-validate whether to which distance to use

ETC3250/5250 Lecture 4

Final implementation

First we standardise the data

```
sim_data2 <- sim_data2 |>
  mutate_if(is.numeric, function(x) (x-mean(x))/sd(x))
```

ETC3250/5250 Lecture 4

Final implementation

Cross-validation in the `tidymodels` `tidymodels` implementation (**https://parsnip.tidymodels.org/reference/nearest_neighbor.html**)

```
tune_spec <-
  nearest_neighbor(
    neighbors = tune(),
    weight_func = tune(),
    dist_power = tune()
  ) |>
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

```
neighbour_grid <- tibble(neighbors = rep(c(7, 9, 11, 13,
                         weight_func = rep(c(rep("rectang
                         dist_power = c(rep(1, 12), rep(2
```

```
set.seed(234)
p_folds <- vfold_cv(sim_data2, v = 5, strata = y)
```

ETC3250/5250 Lecture 4

```r
kNN_wf <- workflow() |>
  add_model(tune_spec) |>
  add_formula(y~.)
```

```r
set.seed(345)
kNN_res <-
  kNN_wf |>
  tune_grid(
    resamples = p_folds,
    grid = neighbour_grid,
    metrics = NULL #NULL to compute a standard set of met
    )
```

```r
kNN_res |>
  collect_metrics() |>
  slice_head(n = 6)
```

```
# A tibble: 6 × 9
  neighbors weight_func dist_power .metric .estimator
mean
      <dbl> <chr>            <dbl> <chr>   <chr>
<dbl>
1         7 inv                  1 accura… binary     0.865
2         7 inv                  1 brier_… binary
```

ETC3250/5250 Lecture 4

```
0.0866
3         7 inv                  1 roc_auc binary     0.931
4         9 inv                  1 accura… binary     0.89
5         9 inv                  1 brier_… binary
0.0829
6         9 inv                  1 roc_auc binary     0.934
# ℹ 3 more variables: n <int>, std_err <dbl>, .config <chr>
```

ETC3250/5250 Lecture 4

Final implementation

```
kNN_res |>
  show_best(metric = "roc_auc", n = 10)
```

```
# A tibble: 10 × 9
   neighbors weight_func dist_power .metric .estimator
mean
       <dbl> <chr>            <dbl> <chr>   <chr>
<dbl>
 1        17 rectangular          1 roc_auc binary
0.951
 2        17 rectangular          3 roc_auc binary
0.950
 3        17 rectangular          2 roc_auc binary     0.95
 4        15 rectangular          2 roc_auc binary
0.946
 5        11 rectangular          1 roc_auc binary
0.946
 6        17 inv                  1 roc_auc binary
0.946
```

ETC3250/5250 Lecture 4

```r
kNN_res |>
  select_best(metric = "roc_auc")
```

```
# A tibble: 1 × 4
  neighbors weight_func dist_power .config
      <dbl> <chr>            <dbl> <chr>
1        17 rectangular          1 Preprocessor1_Model24
```

Then we refit the final model to the full data

```r
knn_final_spec <- nearest_neighbor(neighbors = 17, weight
  set_mode("classification") |>
  set_engine("kknn") #specify how to learn the parameters
```

```r
knn_final_fit <- knn_final_spec |> ## apply the data
  fit(y ~ ., data = sim_data2)
```

ETC3250/5250 Lecture 4

Results

ETC3250/5250 Lecture 4

Logistic regression

ETC3250/5250 Lecture 4

Beyond kNN

- kNN was simple
- fast
- and can give us possibly very complex (wiggly) decision boundaries

However,

- kNN has no way to decide if a feature is important or not
- or give more weight to some features over others
- And while it can produce probabilities for classification…
- it doesn't directly model them

To address these concerns we are going to return to linear modelling

ETC3250/5250 Lecture 4

When linear regression is not appropriate

Consider the following data `Default` in the ISLR R package (textbook) which looks at the default status based on credit balance.

```
library(ISLR)
data(Default)
simcredit <- Default |>
  mutate(default_bin = ifelse(default=="Yes", 1, 0))
simcredit |> slice_head(n=3)
```

```
  default student balance income default_bin
1      No      No     730  44362           0
2      No     Yes     817  12106           0
3      No      No    1074  31767           0
```

Why is a linear model less than ideal for this data?

$\text{default} = \beta_0 + \beta_1 \times \text{balance} + \epsilon$

The linear model predicts values in $\mathbb{R}$ (which can be < 0 or > 1) when our response lives in $\{0, 1\}$

ETC3250/5250 Lecture 4

Modelling binary responses

- Orange line (logistic model fit)
- Is similar to computing a running average of the 0s/1s
- It's much better than the linear fit, because it remains between 0 and 1
- Interpreted as the probability that a certain person defaults

What is a logistic function?

The logistic function

- Instead of predicting the outcome directly,
- We predict the probability of being class 1
- Using the (linear combination of) predictors,
- and the logistic function.

$p(y = 1|\beta_0 + \beta_1 x) = f(x)$
where

$$f(x) = \frac{1}{1 + e^{-(\beta_0+\beta_1 x)}} = \frac{e^{(\beta_0+\beta_1 x)}}{1 + e^{(\beta_0+\beta_1 x)}}$$

ETC3250/5250 Lecture 4

Logistic function

Transform the function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0+\beta_1 x)}}$$

$$\longrightarrow P(y = 1|x) = \frac{1}{1/e^{\beta_0+\beta_1 x}+1}$$

$$\longrightarrow 1/P(y = 1|x) = 1/e^{\beta_0+\beta_1 x} + 1$$

$$\longrightarrow 1/P(y = 1|x) - 1 = 1/e^{\beta_0+\beta_1 x}$$

$$\longrightarrow \frac{1}{1/P(y=1|x)-1} = e^{\beta_0+\beta_1 x}$$

$$\longrightarrow \frac{P(y=1|x)}{1-P(y=1|x)} = e^{\beta_0+\beta_1 x}$$

$$\longrightarrow \log_e \frac{P(y=1|x)}{1-P(y=1|x)} = \beta_0 + \beta_1 x$$

Transforming the probabilities $\log_e \frac{P(y=1|x)}{1-P(y=1|x)}$ makes it possible to see the linear model fit.

> The left-hand side, $\log_e \frac{p}{1-p}$, is known as the log-odds ratio or logit.

ETC3250/5250 Lecture 4

In binary classification $P(y = 0|x) = 1 - P(y = 1|x)$

ETC3250/5250 Lecture 4

The logistic regression model

When there are more than two categories:

- The formula can be extended, using dummy variables (one-hot-encoding).
- If there are K classes, i.e. $\square = \{1, \dots, K\}$ then $y_i = \{0, 0, \dots, 1, \dots, 0\}$ where $y_{ik} = 1$ if y_i was class k
- Introduce parameters β_{0k} and β_{1k} for each class k (excluding class K)
- The probabilities extend from above

$$P(Y = k|x) = \frac{e^{\beta_{0k}+\beta_{1k}x}}{1 + \sum_{j=1}^{K-1} e^{\beta_{0j}+\beta_{1j}x}}, \quad k = 1, \dots K-1$$

$$P(Y = K|x) = \frac{1}{1 + \sum_{j=1}^{K-1} e^{\beta_{0j}+\beta_{1j}x}}$$

ensures the sum of all probabilities is 1.

ETC3250/5250 Lecture 4

Connection to generalised linear models

- To model **binary data**, we need to link our **predictors** to our response using a *link function*.
- Another way to think about it is that we will transform y, to convert it to a proportion, and then build the linear model on the transformed response.
- There are many different types of link functions we could use, but for a binary response we typically use the logistic link function.

ETC3250/5250 Lecture 4

Interpretation

- **Linear regression**
  - y is linear in x
  - β_1 gives the average change in y associated with a one-unit increase in x
- **Logistic regression**
  - y is no longer linear in x becuase of the logistic function
  - β_1 does not correspond to the change in response associated with a one-unit increase in x.
  - But the log-odds are linear in x (logistic regression is still considered a linear model)
  - Increasing x by one unit changes the log odds by β_1 , or equivalently it multiplies the odds by e^{β_1}

Maximum Likelihood Estimation

Now that we have a model for binary data, we need a method to learn its parameters β_0 and β_1

Given the logistic $p(x_i) = P(Y_i = 1|x_i) = \frac{1}{1+e^{-(\beta_0+\beta_1 x_i)}}$ choose parameters β_0, β_1 to maximize the likelihood

The likelihood tells us, how likely was the data that we observed for particular vales of the parameters

$$l_n(\beta_0, \beta_1) := P(Y_1 = y_1, \ldots, Y_n = y_n | x, \beta_0, \beta_1)$$

$$= \prod_{i=1}^{n} P(Y_i = y_i | x, \beta_0, \beta_1)$$

$$= \prod_{i=1}^{n} p(x_i)^{y_i}(1 - p(x_i))^{1-y_i}.$$

We choose the parameter that make the observed data most plausible (i.e. likely)

ETC3250/5250 Lecture 4

Maximum Likelihood Estimation

It is (*almost-always*) more convenient to maximize the *log-likelihood*
The logarithm is a monotonic function so maximising the log-likelihood is the same as maximising the likelihood.

$$\log l_n(\beta_0, \beta_1) = \log \prod_{i=1}^{n} p(x_i)^{y_i}(1 - p(x_i))^{1-y_i}.$$

$$= \sum_{i=1}^{n} \left(y_i \log p(x_i) + (1 - y_i) \log(1 - p(x_i)) \right)$$

$$= \sum_{i=1}^{n} \left(y_i(\beta_0 + \beta_1 x_i) - \log(1 + e^{\beta_0 + \beta_1 x_i}) \right)$$

You will not need to *solve* this yourself, we will use software to find

$$\hat{\beta_0}, \hat{\beta_1} := \arg \max_{\beta_0 \in \mathbb{R}, \beta_1 \in \mathbb{R}^p} \log l_n(\beta_0, \beta_1)$$

ETC3250/5250 Lecture 4

Logistic regression in `tidymodels`

See - **https://parsnip.tidymodels.org/reference/logistic_reg.html**

```
logistic_mod <- logistic_reg() |> #specify the model type
  set_engine("glm") |> #specify how to learn the paramete
  set_mode("classification")
```

```
logistic_fit <- ## apply the data
  logistic_mod |>
  fit(default ~ balance,
      data = simcredit)# fit the model
```

```
tidy(logistic_fit)
```

```
# A tibble: 2 × 5
  term         estimate std.error statistic   p.value
  <chr>           <dbl>     <dbl>     <dbl>     <dbl>
```

ETC3250/5250 Lecture 4

```
1 (Intercept) -10.7      0.361        -29.5 3.62e-191
2 balance       0.00550  0.000220      25.0 1.98e-137
```

Gives the estimated values of $\hat{\beta_0}$ and $\hat{\beta_1}$

ETC3250/5250 Lecture 4

Making predictions

```
simcredit <- simcredit |>
  mutate(p_lr = predict(logistic_fit$fit,
                                simcredit, type = "response"
```

With estimates from the model fit, $\hat{\beta_0}, \hat{\beta_1}$, we can predict the **probability of belonging to class 1** using:

$$p(y = 1|\hat{\beta_0} + \hat{\beta_1}x) = \frac{e^{\hat{\beta_0}+\hat{\beta_1}x}}{1 + e^{\hat{\beta_0}+\hat{\beta_1}x}}$$

Orange points are fitted values, $\hat{p_i}$. Black points are observed response, y_i (either 0 or 1).

ETC3250/5250 Lecture 4

Examine the fit

```
tidy(logistic_fit)
```

```
# A tibble: 2 × 5
  term         estimate std.error statistic   p.value
  <chr>           <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept) -10.7      0.361        -29.5 3.62e-191
2 balance       0.00550  0.000220      25.0 1.98e-137
```

```
glance(logistic_fit)
```

```
# A tibble: 1 × 8
  null.deviance df.null logLik   AIC   BIC deviance
          <dbl>   <int>  <dbl> <dbl> <dbl>    <dbl>
1         2921.    9999  -798. 1600. 1615.    1596.
# ℹ 2 more variables: df.residual <int>, nobs <int>
```

Parameter estimates

$\widehat{\beta}_0 = -10.65$

$\widehat{\beta}_1 = 0.01$

Model fit summary

Null model deviance 2920.6 (error for model with no predictors)
Model deviance 1596.5 (error from fitted model)
These are the extension of the R^2 coefficient form linear models
Think of Model deviance/Null model deviance a bit like $1 - R^2$

ETC3250/5250 Lecture 4

Check the model performance

Compute the roc-auc

```
simcredit$p_lr <- 1- simcredit$p_lr

yardstick::roc_auc(simcredit, default, p_lr)$.estimate
```

```
[1] 0.95
```

```
roc_curve(simcredit, default, p_lr) |>
  autoplot()
```

ETC3250/5250 Lecture 4

Check the model performance

Compute the balanced accuracy based on $\alpha = 0.5$ threshold

```
simcredit_fit <- augment(logistic_fit, simcredit)
simcredit_fit |>
  dplyr::count(default, .pred_class) |>
  group_by(default) |>
  mutate(Accuracy = n[.pred_class==default]/sum(n)) |>
  pivot_wider(names_from = ".pred_class", values_from = n
  select(default, No, Yes, Accuracy)
```

```
# A tibble: 2 × 4
# Groups:   default [2]
  default    No   Yes Accuracy
  <fct>   <int> <int>    <dbl>
1 No       9625    42    0.996
2 Yes       233   100    0.300
```

The accuracy is 0.97, but the balanced accuracy is 0.65.

Unbalanced data set, with very different performance on each class.

How good is this model?

ETC3250/5250 Lecture 4

- Explains about half of the variation in the response, which would normally be reasonable.
- Has excellent roc-auc
- But gets most of the smaller but important class wrong.

ETC3250/5250 Lecture 4

A warning for using GLMs!

> Logistic regression model fitting fails when the data is *perfectly* separated.

MLE fit will try and fit a step-wise function to this graph, pushing coefficients sizes towards infinity and produce large standard errors.

Pay attention to warnings!

```r
logistic_fit <-
  logistic_mod |>
  fit(default_new ~ balance,
      data = simcredit)
```

```
Warning: glm.fit: algorithm did not converge

Warning: glm.fit: fitted probabilities numerically 0 or 1
occurred
```

ETC3250/5250 Lecture 4

Standardisation

- Standardisation is NOT required for logistic regression
- Standardisation is a linear operation (it subtracts and divides by constants)
- and applying linear operations to linear models doesn't change the model
- However, it will help interpretation
- and often make finds the MLE's easier

```
simcredit_std <- simcredit |>
  mutate_if(is.numeric, function(x) (x-mean(x))/sd(x))
```

```
logistic_mod <- logistic_reg() |> #specify the model type
  set_engine("glm") |> #specify how to learn the paramete
  set_mode("classification")
```

```
logistic_fit <- ## apply the data
  logistic_mod |>
```

```
  fit(default ~ balance,
      data = simcredit_std)# fit the model
```

```
simcredit_std <- simcredit_std |>
  mutate(p_lr = predict(logistic_fit$fit,
                                simcredit_std, type = "respo
```

```
tidy(logistic_fit)
```

```
# A tibble: 2 × 5
  term        estimate std.error statistic   p.value
  <chr>          <dbl>     <dbl>     <dbl>     <dbl>
1 (Intercept)    -6.06     0.184     -33.0 1.19e-238
2 balance         2.66     0.107      25.0 1.98e-137
```

ETC3250/5250 Lecture 4

p > 1

- So far we have assumed that $x \in \mathbb{R}$ (i.e. $p = 1$) and therefore that $\beta_1 \in \mathbb{R}$
- Given more than one predictor $x \in \mathbb{R}^p$
- The only change to the logistic regression model is that $\beta_1 x \mapsto \beta^T x = \sum_{j=1}^{p} \beta_j x_j$
- Where $\beta \in \mathbb{R}^p$
- The estimated vector $\hat{\beta}$ is still found using maximum likelihood

ETC3250/5250 Lecture 4

Credit data

ETC3250/5250 Lecture 4

Logistic regression vs k-nearest neighbours

Logistic regression fits a linear decision boundary

$$\frac{1}{1 + \exp(-(\hat{\beta_0} + \hat{\beta}^T x))} = 0.5$$

$$\Rightarrow \hat{\beta_0} + \hat{\beta}^T x = \log\left(\frac{1}{0.5} - 1\right) = 0$$

ETC3250/5250 Lecture 4

Logistic regression vs k-nearest neighbours

Logistic regression fits a linear decision boundary

---

kNN does not

ETC3250/5250 Lecture 4

Logistic regression vs k-nearest neighbours

Logistic regression fits a linear decision boundary

---

kNN does not

ETC3250/5250 Lecture 4

Parametric vs non-parametric

Logistic regression is a parametric machine learning model while kNN is non-parametric

Parametric methods

- Assume that the model takes a specific form
- e.g. $\hat{y}(x) = x^\top \theta$ (linear model) or $\hat{p}(x) = \frac{1}{1+\exp(x^\top \theta)}$ (logistic regression)
- Fitting then is a matter of estimating the parameters of the model, i.e. θ
- Generally considered to be less flexible
- If assumptions are wrong, performance likely to be poor

Non-parametric methods

- No specific assumptions
- Allow the data to specify the model form, without being too rough or wiggly

- More flexible
- Generally needs more observations, and not too many variables
- Easier to over-fit
- e.g. k-Nearest Neighbours or Neural Networks

ETC3250/5250 Lecture 4

Parametric vs non-parametric

From XKCD

ETC3250/5250 Lecture 4

Encoding Categorical Variables

Nice explanation of catergroical encoding with tidymodels

ETC3250/5250 Lecture 4

Categorical Predictors

Both Logistic regression and kNN assume the predictor values are real numbers i.e. $x \in \mathbb{R}^p$

In logistic regression $x_i^\top \beta = \sum_{j=1}^{p} x_j \beta_j$ only defined if x_j is always a number

In kNN $d(x', x_i)$ only defined if x_i is a vector of numbers

Turning categorical variables into numerical variables so that they can be used in these machine learning models is a very important step in effective machine learning.

ETC3250/5250 Lecture 4

Types of Categorical Predictors

Two types categorical variable

- Credit Rating - ranked as `Bad`, `Satisfactory`, `Good` and `Excellent` - is an ordered categroical variable
- State - `VIC`, `NSW`, `SA`, `WA`, `NT` and `TAS` is an unordered categorical variable

Models will not accept categorical variables - encoded as numbers
By default Credit Rating will be encoded as `Bad` = 1, `Satisfactory` = 2, `Good` = 3 and `Excellent` = 4
Ordering of the categories is consistent with the ordering of the encoding, `Excellent` gets more than `Good` which gets more than `Satisfactory` etc.

Unordered Categoprical Predictors

Equally, by default State will be encoded as `VIC` = 1, `NSW` = 2, `SA` = 3, `WA` = 4, `NT` = 5 and `TAS` = 6

But here there was no ordering to the categories, but their encoding has an order

`NSW` gets twice the effect of `VIC` and `TAS` gets 6 times the effect

There is no reason for this in the definition of the variable

This is an *unordered* categorical variable, and an *arbitrary* order is placed on them by the software package

Dummy-encoding/One-hot-encoding

Unordered Categorical variables need to be *encoded*, so as not to impose an arbitrary order

Go from one unordered categorical variable variable $x_j \in \{1, \ldots, K\}$

To K numerical variables $(x_{j_1}, \ldots, x_{j_K}) \in \{0, 1\}^K$ with $\sum_{k=1}^{K} x_{j_k} = 1$

```
# A tibble: 4 × 3
  Income Balance State
   <dbl>   <dbl> <fct>
1    100      20 VIC
2    150      10 TAS
3     80     100 NSW
4    200      70 VIC
```

```
data1_ordered <- data1_unordered |>
  tidytable::get_dummies()

data1_ordered |> slice_head(n=4)
```

```
# A tidytable: 4 × 6
  Income Balance State State_NSW State_TAS State_VIC
   <dbl>   <dbl> <fct>     <int>     <int>     <int>
```

1	100	20	VIC	0	0	1
2	150	10	TAS	0	1	0
3	80	100	NSW	1	0	0
4	200	70	VIC	0	0	1

ETC3250/5250 Lecture 4

What can go wrong in high-dimensions

ETC3250/5250 Lecture 4

Space is huge!

```
set.seed(357)
my_sparse_data <- tibble(cl = c(rep("A", 12),
                                rep("B", 9)),
                         x1 = rnorm(21),
                         x2 = rnorm(21),
                         x3 = rnorm(21),
                         x4 = rnorm(21),
                         x5 = rnorm(21),
                         x6 = rnorm(21),
                         x7 = rnorm(21),
                         x8 = rnorm(21),
                         x9 = rnorm(21),
                         x10 = rnorm(21),
                         x11 = rnorm(21),
                         x12 = rnorm(21),
                         x13 = rnorm(21),
                         x14 = rnorm(21),
                         x15 = rnorm(21)) |>
  mutate(cl = factor(cl)) |>
```

Do we agree that there is no REAL difference between A and B?

Fit a Logistic regression model

▶ Code

```
ggplot(my_sparse_data, aes(x = cl, y = p_lr)) +
 geom_boxplot()
```

Difference is due to having insufficient data with too many variables.

ETC3250/5250 Lecture 4

Regularisation

- We will use regularisation as part of our mdoel fitting process
- Adds a penalty term to the fitting criteria that encourages some of the parameter estimates to be forced to ZERO.
- This effectively reduces the dimensionality by removing noise, - and variability in the sample that is consistent with what would be expected if it was purely noise.

Stay tuned for examples in various methods!

ETC3250/5250 Lecture 4

Next: Decision Trees

ETC3250/5250 Lecture 4